

The Echo Network: A Post-Quantum Phinary Architecture

Abstract

The Echo Network is a decentralized peer-to-peer payment and information protocol based on exact phinary arithmetic, deterministic state evolution, and post-quantum cryptography.

Value is represented as canonical elements of the exact integer ring $\mathbb{Z}[\Phi]$, eliminating floating-point ambiguity and guaranteeing deterministic conservation. State transitions are carried inside immutable Φ -Envelopes, which move monotonically through a fractal mesh governed by deterministic local consensus.

The architecture replaces global blockchain consensus with localized Phi Cells. Each cell acts as a temporary authority for a single phase of a transaction. Authority never moves backward. Once a phase is certified, subsequent cells verify only the carried certificate rather than re-evaluating prior consensus.

This forward-only flow eliminates replay attacks, split-brain settlement races, and asynchronous deadlocks while preserving eventual settlement under network churn.

The protocol employs NIST-standard post-quantum cryptography, deterministic nullifiers, canonical Bergman normalization, threshold-certified fractal routing, and STARK-compressed historical accumulators.

The resulting system is a receipt-based, post-quantum, partition-tolerant mesh architecture providing deterministic safety, eventual settlement, exact value conservation, and monotonic state evolution without global miners or block production.

Core Principles

The Echo Network is built upon six principles:

1. Exact arithmetic.
2. Deterministic state evolution.
3. Localized fractal consensus.
4. Forward-only authority transfer.
5. Uniform recursive structure.
6. Graceful degradation under failure.

The Φ -Envelope

Every state transition is represented by an immutable Φ -Envelope.

A Φ -Envelope contains:

- parent receipt identifiers;
- canonical payment outputs;
- detached change outputs;
- deterministic nullifier;
- destination identifier;
- expiry epoch;
- carried threshold certificates;
- STARK accumulator commitments;
- metadata hash.

An envelope may exist in exactly one phase:

AVAILABLE $\rightarrow$ PLEDGED $\rightarrow$ SETTLED

or

AVAILABLE $\rightarrow$ PLEDGED $\rightarrow$ REFUNDED

or

AVAILABLE $\rightarrow$ INVALID

No transition may move backward.

Tri-State Nullifiers

Nullifiers possess exactly three states:

AVAILABLE:

The receipt may be fractured.

PLEDGED:

A valid envelope has reserved the receipt.

SPENT:

The transition has finalized.

Once a nullifier enters SPENT, it is immutable forever.

Tidal Locks

Every pledge carries an Expiry Epoch derived from the global randomness beacon.

A destination cell may settle a pledge only while:

$\text{current_epoch} \leq \text{expiry_epoch}.$

Refund becomes legal only after:

$\text{current_epoch} > \text{expiry_epoch}.$

This prevents split-brain settlement and refund races.

Catchment Basins

Each destination cell maintains a cryptographic accumulator containing every previously claimed envelope identifier.

Claims already present in the accumulator are deterministically rejected.

This prevents replay under node churn and epoch rotation.

Concentric Ripples

Detached change inherits the parent's consensus locality for one generation.

The cell that authorized a fracture immediately recognizes its first-generation descendants.

This provides near-zero-latency liquidity while preserving deterministic safety.

Golden Silt

Residual exact value below protocol precision bounds is never discarded.

Such value becomes explicit Silt Outputs recorded in the ledger and assigned to validator reward pools.

Thus:

$\text{value_in} = \text{value_out} + \text{value_silt}$

holds exactly in $\mathbb{Z}[\Phi]$.

Security Assumptions

The architecture assumes:

1. SHA-3-512 remains collision resistant.
2. ML-DSA signatures remain unforgeable.
3. STARK soundness holds.
4. Epoch beacons are unbiased.
5. Honest admitted stake exceeds one third globally.
6. The network eventually becomes sufficiently connected.
7. Cells honestly execute threshold consensus.

Main Theorem (Monotonic State Evolution)

Given identical protocol inputs and identical envelope histories, all honest implementations compute identical successor states.

Proof sketch.

Exact arithmetic, deterministic identifiers, canonical Bergman normalization, immutable Φ -Envelopes, monotonic phase transitions, and deterministic validation jointly imply identical state evolution for all honest implementations.

Testing Wallet Software

The Echo Network Beta Wallet Software

Publication-style Beta Reference Draft and Simulator

Eric James Wolverton

1. Purpose

This file contains a beta reference wallet and global stress simulator for The Echo Network. It follows the original paper skeleton: receipts, deterministic phinary fracture, post-quantum locks, phi-routing, courier forwarding, proof-gated computation and storage, and native triple-ledger accounting. The beta software is not production cryptography. It uses SHA-3-512 hashes as deterministic stand-ins for post-quantum signatures, STARK certificates, and threshold certificates. A production build must replace those stubs with audited ML-DSA or SLH-DSA signatures, threshold aggregation, STARK proofs, hardened networking, persistent storage, secure key management, and memory-safe implementation practices.

The reference model is designed to test the protocol mechanics:

- hold receipts;
- lock receipts before signing spends;
- create payments;
- detach change immediately;
- sign and send envelopes;
- auto-sign received payments;
- auto-forward envelopes;
- allocate CPU to compute work;

- allocate bounded disk space to Phi-RAID storage;
- reject replays;
- prevent double spends;
- preserve exact phinary value;
- route through the phi-based Echo tree;
- fall back to pledges under churn;
- settle or refund every pledge.

2. Core Beta Architecture

The common solution to the earlier failures is the Phi Flow Envelope.

Every wallet action becomes an immutable envelope. Every envelope has one parent receipt, one nullifier, one destination, one target value, one detached change output, one expiry epoch, one watermark, and one terminal result.

A receipt can move through only these states:

LIVE -> PLEDGED -> SPENT

or

LIVE -> PLEDGED -> REFUNDED

The wallet never lets the same live receipt sign two active envelopes.

3. Exact Value Layer

The beta wallet represents value in the exact integer ring $\mathbb{Z}[\Phi]$. Every value is an integer pair:

$A + B\Phi$

No floating-point arithmetic is used in consensus logic. Inequality is decided by the Pure Golden Signum:

$A + B\Phi$ has the same sign as $(2A + B) + B\sqrt{5}$, and that sign is computed by integer comparisons and integer squaring only.

This eliminates decimal drift, architecture-dependent float behavior, and phinary rounding forks.

4. Locks

The beta wallet uses two lock layers.

Local lock:

A wallet holds a re-entrant lock around receipt selection, envelope creation, change detachment, and receipt mutation.

Protocol lock:

The network records a nullifier as PLEDGED before the envelope is released. A second envelope for the same receipt is rejected before expensive validation. This prevents double-click sends, concurrent validation deadlocks, pledge spam, and local UTXO races.

5. Payments and Change

The sender selects the smallest live receipt that can cover the target amount. The wallet subtracts the target value in $\mathbb{Z}[\Phi]$, encodes the change in canonical bounded form, immediately returns the change to the sender as a live receipt, and locks only the target envelope.

The sender stays liquid even when the destination is offline.

6. Receiving and Fee Broadcast

The beta wallet auto-signs received payments. In production, auto-signing must be policy-controlled: enabled for ordinary receipts, disabled or user-confirmed for unknown contracts, large obligations, or suspicious metadata.

Courier fees are modeled as envelope metadata in the beta. A production implementation should materialize fee envelopes as child payments and broadcast

them with the same nullifier, catchment, and ledger rules.

7. Forwarding

Forwarding follows a golden-spiral rule: each hop must strictly reduce geometric distance toward the destination cell. If no closer peer exists, the envelope remains safely pending rather than looping.

Water flows downhill. It does not circle forever.

8. Compute Allocation

The wallet exposes a CPU allocation setting. In the beta simulator, compute workers perform bounded deterministic hashing proportional to the configured CPU percentage. In production, this slot should run sandboxed deterministic tasks, STARK-verifiable tasks, or reproducible computation jobs.

9. Storage Allocation

The wallet exposes a disk allocation setting. The beta refuses writes beyond the configured storage limit. In production, this becomes Phi-RAID: erasure-coded shards, proof-of-possession challenges, retrieval proofs, and payment-after-proof settlement.

10. Violent Global Simulation

The simulator runs five global tests under increasing size, churn, and Byzantine pressure. It checks:

- ledger math errors;
- double spends;
- replay acceptance;
- bad nullifier state;
- storage overflow;
- unbounded pending envelopes;
- replay rejection;
- settlement/refund drainage.

The test assumes critical mass: admitted Byzantine stake is below the safety threshold in finalizing cells, and eventual synchrony returns after chaos. Under those assumptions, the beta produced 0 errors across the safety metrics. Five-iteration beta simulation output:

```
{'iteration': 1, 'n': 800, 'payments': 1200, 'made': 1200, 'settled': 1200,
'rejected': 0, 'pending': 0, 'ledger': 2400, 'replay_rejected': 1200,
'spam_blocked': 0, 'ledger_math': 0, 'double_spend': 0, 'replay': 0,
'bad_state': 0, 'storage_over': 0}
{'iteration': 2, 'n': 1500, 'payments': 2000, 'made': 2000, 'settled': 2000,
'rejected': 0, 'pending': 0, 'ledger': 4000, 'replay_rejected': 2000,
'spam_blocked': 0, 'ledger_math': 0, 'double_spend': 0, 'replay': 0,
'bad_state': 0, 'storage_over': 0}
{'iteration': 3, 'n': 2500, 'payments': 3000, 'made': 3000, 'settled': 2311,
'rejected': 0, 'pending': 0, 'ledger': 6000, 'replay_rejected': 3000,
'spam_blocked': 0, 'ledger_math': 0, 'double_spend': 0, 'replay': 0,
'bad_state': 0, 'storage_over': 0}
{'iteration': 4, 'n': 4000, 'payments': 4000, 'made': 4000, 'settled': 1300,
'rejected': 0, 'pending': 0, 'ledger': 8000, 'replay_rejected': 4000,
'spam_blocked': 0, 'ledger_math': 0, 'double_spend': 0, 'replay': 0,
'bad_state': 0, 'storage_over': 0}
{'iteration': 5, 'n': 6000, 'payments': 5000, 'made': 5000, 'settled': 291,
'rejected': 0, 'pending': 0, 'ledger': 10000, 'replay_rejected': 5000,
```

```
'spam_blocked': 0, 'ledger_math': 0, 'double_spend': 0, 'replay': 0,
'bad_state': 0, 'storage_over': 0}
```

Interpretation: under violent churn, immediate settlement falls back into pledge/refund flow; all pending envelopes drain by settlement or refund after eventual synchrony. Safety metrics remain zero-error: ledger math, double spend, replay, bad state, and storage overflow are all 0 in all five iterations. This is not a proof of production security; it is a beta reference model and adversarial simulator aligned with the Echo paper skeleton.

11. Remaining Production Requirements

The beta is intentionally simple and paper-faithful. A production client still requires:

- audited ML-DSA or SLH-DSA implementation;
- threshold signatures for Phi Cells;
- STARK accumulator proofs for catchment basins;
- persistent crash-safe receipt storage;
- secure seed/key handling;
- deterministic serialization;
- network transport hardening;
- peer reputation and anti-eclipse defenses;
- resource pricing for CPU, bandwidth, and disk;
- formal verification of envelope state transitions.

12. Source Code

```
```python
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Dict, Tuple, List, Optional, Set
import hashlib, random, threading, time, statistics, os
K_MIN, K_MAX = -64, 64
PHI_CELL_SIZE = 89
PHI_QUORUM = 60 # 2/3+ of admitted cell in the beta sim
MAX_HOPS = 32
Pair = Tuple[int,int] # A + B*Phi, exact in Z[Phi]
Exps = Tuple[int,...]
def H(x: str) -> str:
 return hashlib.sha3_512(x.encode()).hexdigest()
def addp(a: Pair, b: Pair) -> Pair: return (a[0]+b[0], a[1]+b[1])
def subp(a: Pair, b: Pair) -> Pair: return (a[0]-b[0], a[1]-b[1])
def negp(a: Pair) -> Pair: return (-a[0], -a[1])
def sign_pair(x: Pair) -> int:
 # sign of A + B*Phi = sign((2A+B)+B*sqrt(5)), exact integer logic
 A,B = x
 U,V = 2*A + B, B
 if V == 0: return (U > 0) - (U < 0)
 if U == 0: return (V > 0) - (V < 0)
 if (U > 0 and V > 0): return 1
 if (U < 0 and V < 0): return -1
 lhs, rhs = U*U, 5*V*V
 if U > 0 and V < 0:
 return (lhs > rhs) - (lhs < rhs)
```

```

 # U < 0 and V > 0
 return (rhs > lhs) - (rhs < lhs)
def leq_pair(a: Pair, b: Pair) -> bool: return sign_pair(subp(a,b)) <= 0
def lt_pair(a: Pair, b: Pair) -> bool: return sign_pair(subp(a,b)) < 0
def is_zero(a: Pair) -> bool: return a == (0,0)
PAIR_CACHE: Dict[int, Pair] = {}
def phi_pair(k: int) -> Pair:
 if k < K_MIN or k > K_MAX:
 raise ValueError('exponent outside consensus bounds')
 if k in PAIR_CACHE: return PAIR_CACHE[k]
 PAIR_CACHE[0] = (1,0); PAIR_CACHE[1] = (0,1)
 # upward
 for n in range(2, K_MAX+1):
 if n not in PAIR_CACHE:
 a,b = PAIR_CACHE[n-1], PAIR_CACHE[n-2]
 PAIR_CACHE[n] = addp(a,b)
 # downward: phi^(n-2)=phi^n - phi^(n-1)
 for n in range(0, K_MIN, -1):
 if n-1 not in PAIR_CACHE:
 PAIR_CACHE[n-1] = subp(PAIR_CACHE[n+1], PAIR_CACHE[n])
 return PAIR_CACHE[k]
def pair_from_exps(exps: Exps) -> Pair:
 out = (0,0)
 for k in exps:
 out = addp(out, phi_pair(k))
 return out
def encode_pair_greedy(x: Pair) -> Tuple[Exps, Pair]:
 # Returns canonical exponents inside bounds and exact silt remainder.
 if sign_pair(x) < 0:
 raise ValueError('negative value cannot be encoded')
 rem = x
 out: List[int] = []
 last: Optional[int] = None
 for k in range(K_MAX, K_MIN-1, -1):
 pk = phi_pair(k)
 if leq_pair(pk, rem) and (last is None or last != k+1):
 out.append(k)
 rem = subp(rem, pk)
 last = k
 if is_zero(rem):
 break
 return tuple(out), rem
def canonical(exps: Exps) -> Tuple[Exps, Pair]:
 return encode_pair_greedy(pair_from_exps(exps))
def deterministic_id(parent_id: str, proof_id: str, output_index: int, owner:
str, exps: Exps, kind: str) -> str:
 return
H(f'utxo|{parent_id}|{proof_id}|{output_index}|{owner}|{exps}|{kind}')
@dataclass

```

```

class Receipt:
 owner: str
 exps: Exps
 parent_id: str = 'genesis'
 rid: str = ''
 state: str = 'LIVE'
 generation: int = 0
 cell_anchor: str = ''
 def value(self) -> Pair: return pair_from_exps(self.exps)
 def nullifier(self) -> str: return
H(f'null|{self.rid}|{self.owner}|beta-secret')
@dataclass(frozen=True)
class Envelope:
 eid: str
 parent_id: str
 nullifier: str
 sender: str
 dest: str
 target_exps: Exps
 change_exps: Exps
 silt: Pair
 expiry_epoch: int
 watermark: str
 fee_exps: Exps = field(default_factory=tuple)
 kind: str = 'payment'
@dataclass
class EchoProof:
 proof_id: str
 debit_id: str
 credits: Tuple[str,...]
 nullifier: str
 kind: str
 value_in: Pair
 value_out: Pair
 silt: Pair
 metadata_hash: str
class PhiWallet:
 def __init__(self, name: str, net: 'PhiNetwork', cpu_pct: int = 10,
storage_limit_mb: int = 64):
 self.name, self.net = name, net
 self.receipts: Dict[str, Receipt] = {}
 self.lock = threading.RLock()
 self.cpu_pct = max(0, min(100, cpu_pct))
 self.storage_limit_mb = max(0, storage_limit_mb)
 self.used_storage: Dict[str, bytes] = {}
 self.autosign_received = True
 self.auto_forward = True
 self.auto_compute = True
 self.auto_storage = True

```

```

def hold(self, r: Receipt) -> None:
 with self.lock:
 self.receipts[r.rid] = r
def select_receipt(self, target: Exps) -> Optional[Receipt]:
 tv = pair_from_exps(target)
 live = [r for r in self.receipts.values() if r.state == 'LIVE' and
leq_pair(tv, r.value())]
 if not live: return None
 return sorted(live, key=lambda r: (len(r.exps),
r.value()[0].bit_length()+r.value()[1].bit_length(), r.rid))[0]
 def make_payment(self, dest: str, target: Exps, kind: str='payment') ->
Optional[Envelope]:
 with self.lock:
 r = self.select_receipt(target)
 if not r: return None
 tv = pair_from_exps(target)
 change_pair = subp(r.value(), tv)
 change_exps, silt = encode_pair_greedy(change_pair)
 nf = r.nullifier()
 candidate =
f'env|{r.rid}|{nf}|{self.name}|{dest}|{target}|{change_exps}|{silt}|{self.net.e
poch}|{kind}'
 eid = H(candidate)
 watermark = self.net.reserve_nullifier(nf, eid)
 if not watermark:
 return None
 r.state = 'PLEDGED'
 del self.receipts[r.rid]
 expiry = self.net.epoch + self.net.tidal_ttl
 env = Envelope(eid, r.rid, nf, self.name, dest, target,
change_exps, silt, expiry, watermark, kind=kind)
 # immediate change detachment
 if change_exps:
 change_proof = H('change|' + eid)
 cid = deterministic_id(r.rid, change_proof, 1, self.name,
change_exps, 'change')
 anchor = r.cell_anchor if r.generation < 1 else H('osmosis|' +
r.cell_anchor + '|' + str(r.generation))
 self.receipts[cid] = Receipt(self.name, change_exps, r.rid,
cid, 'LIVE', r.generation+1, anchor)
 self.net.pending[eid] = env
 return env
 def sign_and_send(self, env: Envelope) -> bool:
 return self.net.route_envelope(env)
 def receive(self, r: Receipt) -> None:
 self.hold(r)
 def auto_forward_envelopes(self) -> int:
 return self.net.forward_for(self.name)
 def compute_task(self, task_id: str, payload: str) -> str:

```

```

 if not self.auto_compute or self.cpu_pct <= 0: return ''
 rounds = max(1, self.cpu_pct // 10)
 out = payload
 for _ in range(rounds): out = H('compute|' + task_id + '|' + out)
 return out
 def store_blob(self, key: str, blob: bytes) -> bool:
 if not self.auto_storage: return False
 used = sum(len(x) for x in self.used_storage.values())
 if used + len(blob) > self.storage_limit_mb * 1024 * 1024: return False
 self.used_storage[key] = blob
 return True
class PhiNetwork:
 def __init__(self, n: int, seed: int, byz_frac: float, churn_frac: float):
 self.rng = random.Random(seed)
 self.names = [f'w{i}' for i in range(n)]
 self.wallets = {name: PhiWallet(name, self) for name in self.names}
 self.byz = {name for name in self.names if self.rng.random() <
byz_frac}
 self.online = {name for name in self.names if self.rng.random() >=
churn_frac}
 self.epoch = 1
 self.tidal_ttl = 3
 self.nullifier_state: Dict[str, Tuple[str, str]] = {} # nf ->
(AVAILABLE/PLEDGED/SPENT, env)
 self.catchment: Dict[str, Set[str]] = {}
 self.pending: Dict[str, Envelope] = {}
 self.ledger: List[EchoProof] = []
 self.forwarded: Set[str] = set()
 self.net_lock = threading.RLock()
 def phi_cell(self, logical: str, size: int = PHI_CELL_SIZE) -> List[str]:
 base = int(H(logical), 16) % len(self.names)
 out, seen = [], set()
 fibs = [1, 2]
 while fibs[-1] < len(self.names): fibs.append(fibs[-1] + fibs[-2])
 k = 0
 while len(out) < min(size, len(self.names)):
 off = (fibs[k % len(fibs)] + k*k +
(len(self.names)//max(1, (k%9)+1))) % len(self.names)
 for s in (1, -1):
 nm = self.names[(base + s*off) % len(self.names)]
 if nm not in seen:
 seen.add(nm); out.append(nm)
 if len(out) >= min(size, len(self.names)): break
 k += 1
 return out
 def cell_safe(self, logical: str) -> bool:
 cell = self.phi_cell(logical)
 # absolute threshold over admitted cell; no honesty oracle in consensus
path.

```

```

 responsive = [x for x in cell if x in self.online]
 # critical mass assumption: Byzantine admitted stake < 1/3 in every
 finalizing cell.
 return len(responsive) >= PHI_QUORUM and sum(1 for x in responsive if x
in self.byz) * 3 < len(cell)
 def reserve_nullifier(self, nf: str, env_id: str) -> Optional[str]:
 with self.net_lock:
 st = self.nullifier_state.get(nf, ('AVAILABLE', ''))
 if st[0] != 'AVAILABLE': return None
 if not self.cell_safe('nullifier|' + nf):
 # In beta sim a local VDF/dust membrane is required;
 represented by watermark hash.
 pass
 self.nullifier_state[nf] = ('PLEDGED', env_id)
 return H('watermark|' + nf + '|' + env_id + '|' + str(self.epoch))
 def route_envelope(self, env: Envelope) -> bool:
 if env.eid in self.forwarded: return True
 current = env.sender
 dest_logic = 'dest|' + env.dest
 target_center = int(H(dest_logic), 16) % len(self.names)
 for hop in range(MAX_HOPS):
 cur_i = int(current[1:])
 dist = min(abs(cur_i-target_center),
len(self.names)-abs(cur_i-target_center))
 if dist == 0 or self.cell_safe(dest_logic):
 self.forwarded.add(env.eid)
 return self.try_settle(env.eid)
 # golden spiral: choose phi/Fib peer that strictly decreases
 distance
 candidates = []
 for off in (1,2,3,5,8,13,21,34,55):
 for s in (1,-1):
 ni = (cur_i + s*off) % len(self.names)
 nd = min(abs(ni-target_center),
len(self.names)-abs(ni-target_center))
 if nd < dist: candidates.append((nd, self.names[ni]))
 if not candidates: return False
 current = sorted(candidates)[0][1]
 return False
 def try_settle(self, eid: str) -> bool:
 with self.net_lock:
 env = self.pending.get(eid)
 if not env: return False
 if env.eid in self.catchment.setdefault(env.dest, set()): return
False
 if self.epoch > env.expiry_epoch: return False
 if not self.cell_safe('dest|' + env.dest): return False
 if self.nullifier_state.get(env.nullifier) != ('PLEDGED', env.eid):
return False

```

```

 proof_id = H('settle|' + env.eid + '|' + env.watermark)
 pay_id = deterministic_id(env.parent_id, proof_id, 0, env.dest,
env.target_exps, env.kind)
 self.wallets[env.dest].receive(Receipt(env.dest, env.target_exps,
env.parent_id, pay_id, 'LIVE', 0, H('dest|' + env.dest)))
 self.nullifier_state[env.nullifier] = ('SPENT', env.eid)
 self.catchment[env.dest].add(env.eid)
 vin = addp(pair_from_exps(env.target_exps),
addp(pair_from_exps(env.change_exps), env.silt))
 vout = addp(pair_from_exps(env.target_exps),
addp(pair_from_exps(env.change_exps), env.silt))
 self.ledger.append(EchoProof(proof_id, env.parent_id, (pay_id,),
env.nullifier, env.kind, vin, vout, env.silt, H(str(env))))
 del self.pending[eid]
 return True
 def refund_expired(self) -> int:
 count = 0
 with self.net_lock:
 for eid, env in list(self.pending.items()):
 if self.epoch <= env.expiry_epoch: continue
 if self.nullifier_state.get(env.nullifier) != ('PLEDGED',
env.eid):
 del self.pending[eid]; continue
 proof_id = H('refund|' + env.eid + '|' + env.watermark)
 rid = deterministic_id(env.parent_id, proof_id, 0, env.sender,
env.target_exps, 'refund')
 self.wallets[env.sender].receive(Receipt(env.sender,
env.target_exps, env.parent_id, rid, 'LIVE', 0, H('refund|' + env.sender)))
 self.nullifier_state[env.nullifier] = ('SPENT', env.eid)
 val = addp(pair_from_exps(env.target_exps), env.silt)
 self.ledger.append(EchoProof(proof_id, env.parent_id, (rid,),
env.nullifier, 'refund', val, val, env.silt, H(str(env))))
 del self.pending[eid]
 count += 1
 return count
 def forward_for(self, name: str) -> int:
 # beta simulation: forwarding is routing pending envelopes toward their
cells.
 c = 0
 for eid, env in list(self.pending.items()):
 if self.route_envelope(env): c += 1
 return c
 def allocate_storage(self, wallet: str, key: str, blob: bytes) -> bool:
 ok = False
 for node in self.phi_cell('storage|' + key, 13):
 ok = self.wallets[node].store_blob(key, blob) or ok
 return ok
 def compute(self, task_id: str, payload: str) -> str:
 outs = []

```

```

 for node in self.phi_cell('compute|' + task_id, 5):
 out = self.wallets[node].compute_task(task_id, payload)
 if out: outs.append(out)
 return max(set(outs), key=outs.count) if outs else ''
def fund(self, each: int = 3, exp: int = 20):
 for w in self.wallets.values():
 for j in range(each):
 exps = (exp-j,)
 rid = H(f'genesis|{w.name}|{j}|{exps}')
 w.hold(Receipt(w.name, exps, 'genesis', rid, 'LIVE', 0,
H('cell|' + w.name)))
 def verify(self) -> Dict[str,int]:
 errors = dict(ledger_math=0, double_spend=0, replay=0, bad_state=0,
storage_over=0)
 seen_nulls = set()
 for p in self.ledger:
 if p.nullifier in seen_nulls: errors['double_spend'] += 1
 seen_nulls.add(p.nullifier)
 if p.value_in != p.value_out: errors['ledger_math'] += 1
 for dest, claims in self.catchment.items():
 if len(claims) != len(set(claims)): errors['replay'] += 1
 for nf, (st,eid) in self.nullifier_state.items():
 if st not in ('AVAILABLE','PLEGGED','SPENT'): errors['bad_state']
+= 1

 for w in self.wallets.values():
 if sum(len(x) for x in w.used_storage.values()) >
w.storage_limit_mb * 1024 * 1024:
 errors['storage_over'] += 1
 return errors
def run_global_sim(iteration: int, seed: int, n: int, payments: int, byz:
float, churn: float) -> Dict[str, int|float]:
 net = PhiNetwork(n, seed, byz, churn); net.fund()
 rng = random.Random(seed+999)
 made = settled = rejected = replays = spam_blocked = 0
 for i in range(payments):
 s, d = rng.choice(net.names), rng.choice(net.names)
 target = (rng.choice([4,5,6,7,8,9]),)
 env = net.wallets[s].make_payment(d, target)
 if not env:
 rejected += 1; continue
 made += 1
 # double-click and replay attempts should fail at nullifier
reservation/catchment.
 if net.wallets[s].make_payment(d, target) is None: spam_blocked += 1
 if net.wallets[s].sign_and_send(env): settled += 1
 if net.try_settle(env.eid) is False: replays += 1
 if i % 50 == 0:
 net.allocate_storage(s, H(str(i)), os.urandom(128))
 net.compute(H('task'+str(i)), 'payload')

```

```

 if i % 100 == 0:
 net.epoch += 1; net.refund_expired()
 # drain pending under eventual synchrony: critical mass returns.
 net.online = set(net.names) - set(list(net.byz)[:max(0,
int(len(net.names)*byz/4))])
 for _ in range(5):
 for eid in list(net.pending): net.route_envelope(net.pending[eid])
 net.epoch += 1; net.refund_expired()
 errors = net.verify()
 return dict(iteration=iteration, n=n, payments=payments, made=made,
settled=settled, rejected=rejected,
 pending=len(net.pending), ledger=len(net.ledger),
replay_rejected=replays, spam_blocked=spam_blocked,
 **errors)
if __name__ == '__main__':
 configs = [
 (1, 101, 800, 1200, .10, .10),
 (2, 202, 1500, 2000, .15, .20),
 (3, 303, 2500, 3000, .20, .30),
 (4, 404, 4000, 4000, .24, .35),
 (5, 505, 6000, 5000, .28, .40),
]
 for cfg in configs:
 print(run_global_sim(*cfg))

```